

ANÁLISIS Y DESARROLLO DE SOFTWARE

INTEGRANTES:

ÁNGEL JESÚS HERNÁNDEZ ARÉVALO

RICHARD ALBERTO QUIÑONES QUIÑONES

FICHA 3115426

INSTRUCTORA:

HEIDY LIZBETH ADARME

MEDIAPP

CÚCUTA, NORTE DE SANTANDER

SENA, CIES

2026

MEDIAPP

Sistema web de agendamiento de citas médicas y gestión de historias clínicas para la
optimización de la atención en centros de salud

Presentado por:

ÁNGEL JESÚS HERNÁNDEZ ARÉVALO

RICHARD ALBERTO QUIÑONES QUIÑONES

Ficha 3115426

Trabajo de grado presentado como requisito para optar al título de:

Tecnólogo en Análisis y Desarrollo de Software

Instructora:

Heidy Lizbeth Adarme

SERVICIO NACIONAL DE APRENDIZAJE (SENA)

Centro de la Industria, la Empresa y los Servicios (CIES)

Tecnólogo en Análisis y Desarrollo de Software

CÚCUTA, NORTE DE SANTANDER

2026

Tabla de contenido

RESUMEN	4
ABSTRACT.....	6
INTRODUCCIÓN	8
Capítulo 1. Planteamiento del problema y formulación del proyecto	11
1.1 Descripción del problema	11
1.2 Justificación	12
1.3 Restricciones	13
1.4 Limitaciones.....	14
1.5 Objetivo general.....	15
1.6 Objetivos específicos	15
Capítulo 2. Marco referencial del proyecto	16
2.1 Marco teórico	16
2.2 Marco conceptual.....	20
2.3 Marco tecnológico	22

RESUMEN

El presente proyecto tiene como propósito el análisis, diseño y desarrollo de MediApp, un sistema web de agendamiento de citas médicas y gestión de historias clínicas para centros de salud. La propuesta surge de dos dificultades que aparecen juntas en la atención ambulatoria. La primera es el desorden de la agenda, porque cuando las citas se anotan a mano o en una hoja de cálculo compartida nada impide que dos pacientes queden citados con el mismo médico a la misma hora, que un mismo paciente acumule varias citas en un día o que se agende sobre una fecha que ya pasó. La segunda es la custodia de la información clínica, ya que en un archivo que todos abren no hay forma de sostener que la historia de un paciente solo la escribe el médico que lo atiende.

MediApp responde a ambas con un mismo sistema. Sobre la agenda impone las reglas que la hacen consistente, que son una separación mínima de treinta minutos entre citas del mismo médico, un máximo de una cita por paciente y por día, el rechazo de las fechas ya transcurridas y una reserva del turno que se resuelve dentro de una transacción con bloqueo, de modo que dos solicitudes simultáneas sobre el mismo horario terminen en una cita y un rechazo y nunca en dos citas. Sobre la información clínica reparte el acceso en tres perfiles que no se solapan, y la restricción que le da carácter al proyecto es que el administrador, que puede hacer todo lo demás, no puede crear ni modificar historias clínicas, diagnósticos ni recetas, mientras que el médico, que es quien menos pantallas tiene, es el único que puede hacerlo. El paciente consulta lo que le pertenece y, como única excepción de escritura, reserva y cancela su propia cita desde un calendario de disponibilidad.

El desarrollo se realizó bajo la metodología ágil Scrum, aplicando la encuesta, la entrevista y la observación directa como técnicas de elicitación de requisitos, a partir de las

cuales se especificaron 87 requisitos funcionales, 24 requisitos no funcionales y 60 casos de uso, priorizados con el modelo MoSCoW sobre una escala de valor y urgencia. La solución se construyó con Python y el marco de trabajo Flask, plantillas Jinja2 con Bootstrap y una base de datos MySQL de 11 tablas y 14 claves foráneas. Las contraseñas se almacenan mediante la función de derivación de clave scrypt, el rol se comprueba en el servidor en cada operación y la autoría de todo registro clínico se toma de la sesión y nunca del formulario enviado.

La verificación se realizó mediante una batería de noventa y una pruebas automatizadas que cubre las reglas críticas del sistema, entre ellas la matriz de permisos, el límite de los treinta minutos, el aislamiento de los datos entre pacientes y la carrera entre dos reservas simultáneas. El resultado es un sistema en funcionamiento que ordena la agenda de atención y sostiene, de forma verificable, que el acto clínico pertenece a quien lo ejecuta.

Palabras clave: agendamiento de citas, historia clínica, control de acceso basado en roles, aplicación web, centro de salud, concurrencia.

ABSTRACT

The purpose of this project is the analysis, design, and development of MediApp, a web system for medical appointment scheduling and clinical record management in health centers. The proposal addresses two difficulties that appear together in outpatient care. The first is a disorganized schedule, because when appointments are recorded by hand or in a shared spreadsheet nothing prevents two patients from being booked with the same physician at the same time, a single patient from accumulating several appointments in one day, or an appointment from being booked on a date that has already passed. The second is the custody of clinical information, since in a file that everyone can open there is no way to guarantee that a patient record is written only by the attending physician.

MediApp addresses both within a single system. On the schedule it enforces the rules that keep it consistent, namely a minimum separation of thirty minutes between appointments with the same physician, a maximum of one appointment per patient per day, the rejection of dates that have already passed, and a slot reservation resolved inside a locking transaction, so that two simultaneous requests for the same time result in one appointment and one rejection rather than two appointments. On clinical information it distributes access across three non overlapping profiles, and the restriction that defines the project is that the administrator, who may do everything else, cannot create or modify clinical records, diagnoses, or prescriptions, while the physician is the only role that can. The patient consults what belongs to them and, as the sole writing exception, books and cancels their own appointment from an availability calendar.

Development followed the agile Scrum methodology, using surveys, interviews, and direct observation as requirements elicitation techniques, from which 87 functional requirements, 24 non functional requirements, and 60 use cases were specified and prioritized with the

MoSCoW model. The solution was built with Python and the Flask framework, Jinja2 templates with Bootstrap, and a MySQL database of 11 tables and 14 foreign keys. Passwords are stored using the scrypt key derivation function, the user role is verified on the server for every operation, and the authorship of any clinical record is taken from the session and never from the submitted form.

Verification was carried out through a suite of ninety one automated tests covering the critical rules of the system, including the permission matrix, the thirty minute limit, data isolation between patients, and the race condition between two simultaneous bookings. The result is a working system that organizes the care schedule and verifiably upholds that the clinical act belongs to whoever performs it.

Keywords: appointment scheduling, clinical record, role based access control, web application, health center, concurrency.

INTRODUCCIÓN

La atención ambulatoria de un centro de salud pequeño se organiza alrededor de una agenda, y de lo bien que esa agenda se lleve depende buena parte de lo que el paciente percibe como calidad del servicio. Una cita mal anotada no es un error administrativo aislado, porque se traduce en una sala de espera con dos personas citadas a la misma hora, en un médico que atiende con retraso el resto de la jornada y en un paciente que vuelve a su casa sin haber sido atendido.

El diagnóstico que dio origen a este proyecto encontró que la dificultad no está en la falta de voluntad de quien agenda, sino en la herramienta con que lo hace. Una hoja de cálculo compartida, o un cuaderno, registran lo que se les escribe y no comprueban nada. No advierten que el médico ya tiene una cita quince minutos antes, no impiden que el mismo paciente quede citado tres veces en la misma semana y no distinguen entre una fecha futura y una que ya pasó. La consecuencia es que la consistencia de la agenda depende por completo de la memoria de una persona, y esa memoria falla el día que hay más trabajo, que es justamente cuando más importa.

Junto a la agenda aparece una segunda dificultad, de naturaleza distinta y de mayor consecuencia. La información clínica de un paciente, que es su historia, su diagnóstico y lo que se le prescribe, tiene un responsable con nombre propio, que es el médico que lo atendió. Cuando esa información vive en un archivo que todo el personal puede abrir y modificar, esa responsabilidad deja de poder sostenerse, porque nada distingue lo que escribió el médico de lo que corrigió después alguien más. No es un problema de desconfianza hacia el personal administrativo, sino de que un registro clínico sin autoría verificable no sirve como registro clínico.

En respuesta a esa situación surge MediApp, un sistema web que atiende las dos dificultades con un mismo diseño. La agenda deja de ser un registro pasivo y pasa a comprobar, antes de guardar cualquier cita, que el médico tenga el tiempo libre que necesita, que el paciente no tenga ya otra cita ese día y que la fecha no haya transcurrido. La información clínica deja de estar al alcance de todos y queda repartida en tres perfiles de acceso que el servidor comprueba en cada operación, de manera que la restricción no dependa de que una pantalla oculte un botón.

El reparto de esos tres perfiles es lo que distingue a este proyecto de un sistema de gestión corriente, y conviene enunciarlo desde el principio porque el resto del documento se apoya en él. El administrador gestiona la operación completa, es decir los usuarios, los médicos, los pacientes, los catálogos y la agenda, y su única restricción es que no puede crear ni modificar historias clínicas, diagnósticos ni recetas. El médico, que es el rol con menos pantallas, es el único que puede hacerlo, y en cambio consulta su agenda sin poder modificarla. El paciente consulta lo que le pertenece y tiene una sola excepción de escritura, que es reservar y cancelar su propia cita desde un calendario de disponibilidad.

El desarrollo se fundamentó en conceptos relacionados con el agendamiento de la atención, la custodia del registro clínico, el control de acceso basado en roles y las funciones de derivación de clave para el almacenamiento de contraseñas. Se aplicó la metodología ágil Scrum, con la encuesta, la entrevista y la observación directa como técnicas de elicitación, y la construcción se realizó con Python, el marco de trabajo Flask y una base de datos MySQL.

El documento se organiza en seis capítulos. El primero plantea el problema, la justificación y los objetivos. El segundo reúne el marco teórico, conceptual y tecnológico. El tercero describe la metodología de investigación y de desarrollo. El cuarto presenta el análisis y la especificación de requisitos, con su priorización y los riesgos del proyecto. El quinto expone el

diseño de la solución, la arquitectura, el modelo de control de acceso y el modelo de datos. El sexto relata el desarrollo, la implementación, las pruebas y los resultados obtenidos. El documento cierra con las conclusiones, las referencias y los anexos.

Capítulo 1. Planteamiento del problema y formulación del proyecto

1.1 Descripción del problema

La atención ambulatoria de un centro de salud pequeño se organiza alrededor de una agenda, y de la consistencia de esa agenda depende buena parte de lo que el paciente percibe como calidad del servicio. Cuando las citas se registran en un cuaderno o en una hoja de cálculo compartida, el registro anota lo que se le escribe y no comprueba nada, de manera que la consistencia queda encomendada por completo a la memoria de quien agenda.

El diagnóstico realizado sobre la operación de centros de atención de este tamaño identificó tres situaciones que se repiten y que tienen la misma causa. La primera es el choque de horarios, porque nada impide citar a dos pacientes con el mismo médico a la misma hora ni dejar entre dos citas un margen menor del que dura la consulta. La segunda es la acumulación, ya que un mismo paciente puede quedar citado varias veces en el mismo día sin que el registro lo advierta. La tercera es el agendamiento sobre fechas ya transcurridas, que ocurre al copiar una programación anterior sin revisarla.

Las consecuencias de esas tres situaciones no se quedan en el mostrador. Una sala de espera con dos personas citadas a la misma hora produce un retraso que el médico arrastra el resto de la jornada, y termina en un paciente que regresa a su casa sin haber sido atendido. El costo no es solamente de tiempo, porque una cita perdida en un servicio de salud puede significar un control que no se hizo o un tratamiento que se demoró.

Junto a la agenda aparece una segunda dificultad, de naturaleza distinta y de mayor consecuencia. La información clínica de un paciente, que es su historia, su diagnóstico y lo que se le prescribe, tiene un responsable con nombre propio, que es el médico que lo atendió. Cuando esa información vive en un archivo que todo el personal puede abrir y modificar, la

responsabilidad deja de poder sostenerse, porque nada distingue lo que escribió el médico de lo que corrigió después alguien más. No se trata de desconfianza hacia el personal administrativo, sino de que un registro clínico sin autoría verificable pierde su condición de registro clínico.

Las dos dificultades comparten una misma raíz, y es que la herramienta con la que se trabaja no distingue entre quién hace qué. Un cuaderno no sabe qué es una regla de agenda y una hoja de cálculo no sabe qué es un rol, de modo que ni la consistencia de la programación ni la custodia del acto clínico pueden apoyarse en ellas. De ahí que el problema no se resuelva ordenando mejor el archivo, sino sustituyéndolo por un sistema que compruebe.

1.2 Justificación

MediApp se justifica porque atiende las dos dificultades anteriores con un mismo diseño y en el punto exacto donde se originan. Sobre la agenda convierte el registro pasivo en una comprobación, de modo que ninguna cita llega a guardarse sin que el sistema haya verificado que el médico dispone del tiempo necesario, que el paciente no tiene ya otra cita ese día y que la fecha no ha transcurrido. La regla deja de depender de que alguien la recuerde el día de más trabajo, que es justamente cuando se olvida.

Sobre la información clínica, el sistema reparte el acceso en tres perfiles y comprueba el perfil en el servidor en cada operación, no ocultando opciones en la pantalla. La diferencia entre las dos formas de hacerlo es la que separa una restricción real de una apariencia de restricción, porque una opción oculta vuelve a mostrarse desde el navegador mientras que una comprobación en el servidor rechaza la petición aunque llegue por fuera de la interfaz.

El reparto de esos perfiles es lo que distingue a este proyecto de un sistema de gestión corriente. El administrador gestiona la operación completa y su única restricción es que no puede crear ni modificar historias clínicas, diagnósticos ni recetas. El médico, que es el perfil con

menos pantallas, es el único que puede hacerlo. Esa inversión no es un detalle de configuración, porque traslada al sistema una regla del ejercicio profesional, y es que el acto clínico pertenece a quien lo ejecuta.

El beneficio se reparte entre los tres perfiles y es distinto en cada uno. La administración deja de sostener la agenda con la memoria y gana un registro que rechaza lo que no puede ser. El médico obtiene la certeza de que lo que firma queda a su nombre y no se altera después. El paciente deja de depender de una llamada en horario de oficina para pedir o cancelar su cita, porque lo hace desde un calendario que le muestra los horarios libres sin revelarle quién ocupa los demás.

Desde el punto de vista técnico, el proyecto se justifica además como ejercicio de formación sobre un problema que no admite soluciones aparentes. Una agenda compartida obliga a resolver la concurrencia, porque dos personas pueden pedir el mismo horario en el mismo instante, y un sistema descuidado se lo concede a las dos. Un reparto de acceso por roles obliga a distinguir entre esconder y prohibir. Ambas cuestiones se resuelven en este trabajo y se verifican con pruebas automatizadas, que es lo que permite afirmar que están resueltas en lugar de suponerlo.

1.3 Restricciones

Las restricciones son condiciones impuestas al proyecto desde fuera, que delimitan lo que puede construirse y bajo qué condiciones.

- El proyecto se desarrolla como trabajo de grado, de modo que su alcance y su plazo son los del programa de formación y no los de un producto comercial.
- El equipo lo componen dos personas que cursan al mismo tiempo el programa, sin dedicación exclusiva al desarrollo.

- La infraestructura disponible es un entorno local de desarrollo, sin servidor contratado ni dominio propio, por lo que el cifrado del tránsito depende de dónde se despliegue el sistema y no del programa.
- Se emplean únicamente herramientas libres y de uso común, sin componentes de licencia propietaria, para que la solución pueda instalarse sin costo de licenciamiento.
- El sistema no puede presentarse como historia clínica electrónica certificada, porque su habilitación exige requisitos normativos y de interoperabilidad que exceden el alcance académico del proyecto.
- Los datos con los que se demuestra el funcionamiento son ficticios, ya que operar con información clínica real exigiría autorización del titular y un tratamiento de datos personales que el proyecto no está en condiciones de garantizar.

1.4 Limitaciones

Las limitaciones son alcances que la solución no cubre. Se declaran de forma expresa porque un sistema que custodia información clínica de personas identificadas no admite ambigüedad sobre lo que hace y lo que no, y porque un documento que solo describe lo que el sistema tiene deja al lector suponiendo el resto.

- El sistema no notifica la cita por correo electrónico ni por mensaje de texto, de modo que el recordatorio sigue dependiendo del paciente.
- No incluye teleconsulta ni videollamada, porque pertenecen a la prestación del servicio y no a su agendamiento, que es lo que el objetivo general delimita.
- No factura los servicios prestados ni administra convenios con aseguradoras.
- No firma digitalmente la historia clínica, lo que requeriría una autoridad certificadora.

- No realiza respaldos automáticos ni aplica una política de retención, pues son funciones de la administración del servidor de base de datos y no del sistema.
- No cuenta con aplicación móvil nativa, aunque la interfaz responde desde el navegador de un teléfono.

Ninguna de estas limitaciones es un olvido. Todas corresponden a capacidades que se evaluaron durante el análisis y se descartaron para esta versión, y aparecen en el anexo de especificación de requisitos como la categoría de aquello que de forma expresa no se construirá.

1.5 Objetivo general

Desarrollar MediApp, un sistema web de agendamiento de citas médicas con acceso diferenciado para administrador, médico y paciente, que permita gestionar la disponibilidad de la atención médica y controlar el historial clínico en centros de salud.

1.6 Objetivos específicos

1. Facilitar la programación de la atención médica mediante un módulo interactivo que permita al administrador y al paciente agendar citas de forma intuitiva.
2. Habilitar la gestión del acto clínico como responsabilidad exclusiva del médico tratante para la creación y actualización de historias clínicas, diagnósticos y prescripciones.
3. Garantizar la confidencialidad de la información médica mediante un control de acceso basado en roles que restrinja la visualización y edición según el perfil del usuario.

Capítulo 2. Marco referencial del proyecto

2.1 Marco teórico

El marco teórico reúne los fundamentos sobre los que se sostiene la solución propuesta, esto es, la naturaleza reservada de la historia clínica, la programación de la atención entendida como una asignación de recursos sujeta a restricciones, el control de la concurrencia sobre un recurso compartido, el modelo de acceso basado en roles y el almacenamiento seguro de las credenciales.

La historia clínica como documento reservado. La historia clínica no es un archivo administrativo más. La normativa colombiana la define como un documento privado, sometido a reserva, cuyo diligenciamiento corresponde únicamente al personal que interviene de forma directa en la atención del paciente (Ministerio de Salud, 1999). A esa reserva se suma el régimen general de protección de datos personales, que clasifica los datos relativos a la salud como datos sensibles y sujeta su tratamiento a condiciones más estrictas que las de un dato corriente (Ley 1581 de 2012). De ahí se desprende la consecuencia técnica que atraviesa este proyecto, y es que la autoría de un registro clínico y el alcance de quien puede leerlo no son preferencias de configuración sino condiciones que el sistema debe sostener por diseño.

El agendamiento como asignación de recursos con restricciones. Programar la atención de un centro de salud consiste en asignar un recurso escaso, que es el tiempo del profesional, a un conjunto de solicitudes que compiten por él. El problema no está en registrar la asignación sino en verificar que cumple las restricciones del dominio, entre ellas que un profesional no puede atender dos consultas a la vez, que entre dos atenciones debe mediar el tiempo que la consulta dura y que no puede programarse una atención sobre un instante que ya

transcurrió. Un registro que se limita a guardar lo que se le escribe traslada esa verificación a la persona que agenda, y con ella la posibilidad de equivocarse.

Concurrencia sobre un recurso compartido. Cuando varias personas pueden reservar el mismo recurso, aparece una dificultad que no existe cuando trabaja una sola. Dos solicitudes simultáneas pueden consultar la disponibilidad antes de que cualquiera de las dos escriba, de modo que ambas encuentran el horario libre y ambas lo reservan. La teoría de bases de datos describe esta situación entre las anomalías que el control de concurrencia debe evitar, y ofrece como solución que la comprobación y la escritura ocurran dentro de una misma transacción que mantenga bloqueadas las filas consultadas hasta confirmar (Elmasri y Navathe, 2016). La consecuencia de diseño es que la validación de la agenda no puede resolverse consultando primero y guardando después, porque entre ambas operaciones cabe otra petición.

Control de acceso basado en roles. El modelo de control de acceso basado en roles propone que los permisos no se asignen a las personas sino a los roles que estas desempeñan, y que las personas los reciban por pertenecer a un rol (Sandhu et al., 1996). La propuesta de estandarización posterior formaliza sus elementos y sus relaciones, y precisa que la asignación de permisos es una decisión de la organización y no del usuario (Ferraiolo et al., 2001). Su ventaja frente a conceder permisos uno a uno es que el permiso queda enunciado sobre la función y no sobre el individuo, de modo que incorporar a un profesional nuevo consiste en asignarle un rol y no en revisar una lista de autorizaciones.

La diferencia entre ocultar y prohibir. Un sistema puede dejar de mostrar una opción a quien no debe usarla, y esa es una decisión de interfaz. Puede además rechazar la operación cuando llega, y esa es una decisión de autorización. Las dos se ven igual mientras el usuario se comporte como se espera, pero se distinguen en cuanto la petición se envía por fuera de la

interfaz, porque una opción oculta vuelve a estar disponible y una comprobación del servidor no. Por esa razón el control de acceso se considera una propiedad del servidor, y lo que la interfaz hace es acompañarla, no sustituirla.

Almacenamiento de contraseñas y funciones de derivación de clave. Una contraseña no se guarda, se deriva. El sistema almacena únicamente el resultado de aplicarle una función de derivación de clave, y comprobarla consiste en volver a derivar la que el usuario escribe y comparar los dos resultados, nunca en descifrar lo almacenado. Estas funciones se diseñan de forma deliberada para ser costosas, porque de ese costo depende cuánto tarda quien intente adivinar contraseñas a partir de una base de datos sustraída.

Por qué `scrypt` y no `bcrypt`. Las dos funciones más difundidas para esta tarea responden a amenazas de épocas distintas. La primera de ellas se diseñó para que su costo pudiera aumentarse con el tiempo mediante un parámetro, de modo que el avance del hardware no la volviera obsoleta (Provos y Mazières, 1999). Su costo, sin embargo, es fundamentalmente de cómputo, y el cómputo es precisamente lo que resulta barato de paralelizar en tarjetas gráficas y en circuitos dedicados.

La función empleada en este proyecto pertenece a una generación posterior, cuyo planteamiento consiste en exigir además una cantidad configurable de memoria durante el cálculo (Percival, 2009). La memoria es un recurso caro de multiplicar en el hardware especializado, de manera que un atacante que quiera probar miles de contraseñas en paralelo necesita miles de veces esa memoria, y el ataque deja de abaratarse simplemente añadiendo unidades de cálculo. La función quedó posteriormente especificada como estándar de Internet (Percival y Josefsson, 2016), lo que la sitúa entre las opciones documentadas y no entre las soluciones improvisadas.

Ambas funciones se consideran aceptables para el almacenamiento de contraseñas en las recomendaciones vigentes de seguridad en aplicaciones web, que las enumeran junto a otras derivadas del mismo principio y desaconsejan en cambio el uso de funciones de resumen de propósito general, que no fueron diseñadas para esta tarea (OWASP, 2021). La elección de una función con endurecimiento de memoria responde, por tanto, a una diferencia de propiedades frente a un atacante con hardware especializado, y no a una preferencia de implementación. En el apartado 6.5 se describe cómo queda aplicada dentro del sistema.

Calidad del software y verificación. La calidad de un producto de software se describe mediante atributos que pueden evaluarse por separado, entre ellos la funcionalidad, la fiabilidad, la usabilidad, la seguridad y la mantenibilidad (ISO/IEC, 2011). Un atributo enunciado sin una forma de comprobarlo no es un requisito sino una aspiración, razón por la cual la especificación de este proyecto acompaña cada requisito no funcional con el modo en que se verifica. La verificación automatizada añade una propiedad que la manual no tiene, y es que puede repetirse íntegra después de cada cambio sin depender de que alguien recuerde hacerlo (Pressman y Maxim, 2020).

Ingeniería de requisitos. La especificación de requisitos distingue entre lo que el sistema debe hacer y las condiciones bajo las cuales debe hacerlo, y advierte que los defectos introducidos en esta etapa son los más costosos de corregir, porque se propagan a todo lo que se construye después (Sommerville, 2011). De ese principio se desprende la organización del capítulo 4, donde los requisitos se enuncian, se priorizan y se hacen trazables hasta los casos de uso y hasta las tablas sobre las que operan.

2.2 Marco conceptual

Se relacionan a continuación los términos empleados a lo largo del documento, con el significado preciso que tienen dentro del dominio de la atención en salud y de este proyecto en particular.

- **Historia clínica:** registro cronológico de las condiciones de salud de un paciente y de los actos que sobre él se realizan. Es un documento reservado, y su diligenciamiento corresponde al personal que interviene directamente en la atención.
- **Acto clínico:** conjunto de registros que el profesional produce sobre un paciente al atenderlo, que en este sistema son la historia clínica, el diagnóstico con su plan de tratamiento y la prescripción.
- **Cita médica:** reserva de una franja de tiempo de un profesional para atender a un paciente determinado, con una fecha, una hora y un motivo.
- **Disponibilidad:** conjunto de franjas de tiempo de un profesional que no están ocupadas por otra cita ni alcanzadas por la separación mínima que debe existir entre dos atenciones.
- **Rol:** conjunto de permisos que se concede a una función dentro de la organización y no a una persona concreta. Las personas reciben los permisos por pertenecer al rol.
- **Control de acceso basado en roles:** modelo de autorización en el que los permisos se asignan a roles y los roles a los usuarios, de modo que cambiar lo que alguien puede hacer se resuelve cambiándole el rol y no revisando permiso por permiso.
- **Sesión:** estado que el sistema conserva entre peticiones para recordar quién está autenticado y con qué rol, y del que se toma la identidad de quien firma un registro.

- **Función de derivación de clave:** algoritmo que transforma una contraseña en un valor almacenable del que no puede recuperarse la original, diseñado de forma deliberada para ser costoso de calcular.
- **Hash de contraseña:** resultado de aplicar una función de derivación de clave a una contraseña. Es lo único que el sistema almacena, y comprobar una contraseña consiste en derivarla de nuevo y comparar, nunca en descifrar lo guardado.
- **Transacción:** conjunto de operaciones sobre la base de datos que se confirman todas o no se confirma ninguna, de manera que una falla intermedia no deja registros a medias.
- **Condición de carrera:** situación en la que dos operaciones simultáneas leen el mismo estado antes de que cualquiera escriba, y ambas concluyen que pueden actuar cuando solo una debería.
- **Bloqueo:** mecanismo por el cual una transacción impide que otra lea o modifique las filas que está a punto de cambiar, hasta que confirme o deshaga.
- **Baja lógica:** retiro de un registro de la operación cambiando su estado en lugar de borrarlo, de modo que lo que lo referencia siga siendo consultable.
- **Clave foránea:** columna que referencia la clave primaria de otra tabla, con lo que la base impide que quede apuntando a una fila que no existe.

2.3 Marco tecnológico

Se describen las tecnologías empleadas en la construcción de la solución, indicando la función que cumple cada una dentro del sistema. La selección respondió a tres condiciones, que son la disponibilidad de herramientas libres, la posibilidad de instalar el conjunto en un equipo corriente sin costo de licenciamiento y la conveniencia formativa de emplear componentes cuyo funcionamiento quede a la vista.

- **Python:** lenguaje de programación interpretado y de tipado dinámico, con el que se construyó la totalidad de la lógica del servidor.
- **Flask:** marco de trabajo web para Python que resuelve el enrutamiento de las peticiones, la gestión de la sesión y el renderizado de las plantillas. Se eligió por ser un marco reducido, que no impone una estructura de proyecto y deja a la vista lo que ocurre en cada petición, lo que resulta apropiado para un trabajo cuyo propósito incluye comprender el mecanismo y no solo usarlo.
- **Jinja2:** motor de plantillas con el que el servidor arma el HTML antes de enviarlo al navegador, de manera que la decisión de qué mostrar a cada rol se toma en el servidor.
- **Bootstrap:** biblioteca de estilos que proporciona la rejilla y los componentes de la interfaz, y con la que las pantallas se adaptan al ancho del dispositivo.
- **MySQL:** sistema gestor de bases de datos relacionales que almacena las once tablas del esquema. Aporta las claves foráneas que sostienen la integridad y el bloqueo de filas con el que se resuelve la reserva simultánea de un mismo horario.
- **mysql-connector-python:** conector oficial que comunica la aplicación con el gestor de base de datos, empleado con consultas parametrizadas y con un conjunto reutilizable de conexiones.

- **werkzeug.security**: biblioteca que acompaña a Flask y que proporciona la derivación y la verificación de las contraseñas mediante script.
- **pytest**: herramienta con la que se escribieron y se ejecutan las pruebas automatizadas que verifican las reglas críticas del sistema.
- **Git**: sistema de control de versiones con el que se registró la evolución del proyecto.

El desarrollo se condujo con la metodología ágil Scrum, cuyos elementos y ciclo se describen en el capítulo 3 (Schwaber y Sutherland, 2020).